

COMPASS_Synop- tic_LE_Tree_Flux_Calcs_2023_CH4

Roberta Peixoto

2026-04-29

Table of contents

1	Add Required Packages	2
2	Pull in Raw Datafiles	2
3	Read in metadata file and match to raw data	4
4	Check start times overlaps or missing values	5
5	Match metadata to raw data	6
6	Reading matched data and metadata	8
7	Reading Temp df to merge to metadata	10
8	fixing some Deadbands and stop time	12
9	Calculate fluxes	13
10	check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)	20
11	Spot check a few fluxes with $R^2 > 0.9$	21
12	Take a look at the fluxes < 0.9 and a flux estimate below 1	24
13	Take a look at the bad fluxes < 0.9 and have a high flux estimate	27
14	See if we want to remove these fluxes or if we want to try and recalculate these ones?	27

15 Look at the really high or really low ones to see if there is a timing issue or something	30
16 Check all fluxes	32
17 QAQC - Flags	36
18 Read out final dataframe and export for use elsewhere	42

1 Add Required Packages

2 Pull in Raw Datafiles

```
#defining where the raw data is

DATA_DIR_ROOT <- here::here(
  "Tree_Fluxes_Synoptic_LE",
  "2023",
  "Raw Data"
)

#fucntion to read data
now <- function() format(Sys.time(), "%Y-%m-%d %H:%M:%S")
message(now(), " | Script started")
#finding the zip files
message(now(), " | Looking for ZIP files in: ", DATA_DIR_ROOT)

zip_files <- list.files(
  DATA_DIR_ROOT,
  pattern = "\\\\.zip$",
  full.names = TRUE)

message(now(), " | ZIP files found: ", length(zip_files))
print(zip_files)
if (length(zip_files) == 0) {
  stop("No ZIP files found. Check DATA_DIR_ROOT.")}

cache <- list()
```

```

# Reader function
read_li7810_zip <- function(zip_file, tz = "US/Eastern") {
  message(now(), " | Processing ZIP: ", basename(zip_file))
  if (zip_file %in% names(cache)) {
    message(" -> Using cached version")
    return(cache[[zip_file]]) }

  tmp_dir <- tempfile("li7810_")
  dir.create(tmp_dir)
  on.exit(unlink(tmp_dir, recursive = TRUE), add = TRUE)
  unzip(zip_file, exdir = tmp_dir)
  data_files <- list.files(
    tmp_dir,
    pattern = "\\\\.data$",
    full.names = TRUE,
    recursive = TRUE )

  message(" -> .data files found: ", length(data_files))
  if (length(data_files) == 0) {
    stop("No .data files inside ZIP: ", zip_file) }

  dat_list <- lapply(data_files, function(f) {
    message("   Reading: ", basename(f))
    ffi_read_LI7810(f)})

  dat <- data.table::rbindlist(dat_list, fill = TRUE) %>%
    mutate(TIMESTAMP = with_tz(TIMESTAMP, tz))
  cache[[zip_file]] <- dat
  message(" -> ZIP finished: ", basename(zip_file))
  return(dat)}

# ACTUAL EXECUTION
message(now(), " | Starting data read")
alldat_list <- lapply(zip_files, read_li7810_zip)
alldat <- data.table::rbindlist(alldat_list, fill = TRUE)
message(now(), " | DONE")
message("Final number of rows: ", nrow(alldat))

# Show result
print(head(alldat))

```

3 Read in metadata file and match to raw data

```
md <- "Raw Data/COMPASS_SynopticLE_TreeFlux_Metadata_2023.csv"
dat <- read.csv(md)

metadat <- dat %>%
  # Remove rows without date
  filter(!is.na(Date)) %>%
  # Keep only desired instruments
  filter(SN %in% c("TG10-01448", "TG10-01426", "TG10-01028")) %>%
  # Normalize Plot labels
  mutate(
    Plot_raw = toupper(Plot),
    # Extract plot components
    #Site = str_extract(Plot_raw, "[A-Z]{3}"),
    #Zone = str_extract(Plot_raw, "(TR|UP)"), # somehow all of the PTR measurements turned in
    Site = toupper(Site),
    Zone = toupper(Zone),
    Replicate = str_extract(Plot_raw, "(?<=(TR|UP))[0-9]+") |> as.integer(),
    # Letter ONLY if it comes AFTER a number
    Letter = str_extract(Plot_raw, "(?<=[0-9])[A-Z]$"),
    # Time normalization
    Time_start = str_replace_all(as.character(Time_start), "\\.", ":") |> str_trim(),
    Stop.time = str_replace_all(as.character(Stop.time), "\\.", ":") |> str_trim(),
    # Safe time parsing (NO WARNINGS)
    t_start = suppressWarnings(parse_date_time(Time_start, orders = c("H:M:S", "H:M"))),
    t_stop = suppressWarnings(parse_date_time(Stop.time, orders = c("H:M:S", "H:M"))),
    # Observation length
    Obs_length = as.numeric(t_stop - t_start, units = "secs"),
    # Timestamp
    TIMESTAMP = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    # Sequential ID
    Seq = row_number(),
    # INAL standardized Plot label
    Plot = if_else(
      is.na(Letter),
      paste(Date, Site, Zone, Replicate, Seq, sep = "_"),
```

```

      paste(Date, Site, Zone, Replicate, Letter, Seq, sep = "_")) )

dplyr::count(metadat, Site, Zone)

# Quick check
head(metadat$Plot)

```

4 Check start times overlaps or missing values

```

#checking if there is any start and stop time overlap from the field notes
overlaps_detected <- metadat %>%
  mutate(
    start_ts = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    end_ts = start_ts + Obs_length
  ) %>%
  arrange(SN, start_ts) %>%
  group_by(SN) %>%
  mutate(
    next_plot      = lead(Plot),
    next_start_ts  = lead(start_ts),
    overlap_fffi   = !is.na(next_start_ts) & next_start_ts <= end_ts
  ) %>%
  filter(overlap_fffi) %>%
  select(
    SN,
    Plot,
    start_ts,
    end_ts,
    next_plot,
    next_start_ts,
    Obs_length
  )

print(overlaps_detected)

```

```

#checking problems with obs_length - this is a frequent bug for manual field notes
field_notes_overlap<-metadat %>%
  mutate(
    obs_problem =
      is.na(Obs_length) | Obs_length <= 0
  ) %>%
  filter(obs_problem) %>%
  select(
    SN,
    Date,
    Plot,
    Time_start,
    Stop.time,
    Obs_length
  )

print(field_notes_overlap)

# checking unique measurment start time and stop time
# alldat %>%
#   filter(
#     SN == "TG10-01448",
#     TIMESTAMP > as.POSIXct("2023-01-05 13:28:00"),
#     TIMESTAMP < as.POSIXct("2023-01-05 13:33:00")
#   ) %>%
#   ggplot(aes(x = TIMESTAMP, y = CO2)) +
#   geom_point()#ylim(300,500)

```

5 Match metadata to raw data

```

#function to match data per equipment (considering different serial numbers) uhul!
match_flux_metadata <- function(alldat, metadat) {
  message("Starting metadata Licor matching")
  #Sanity checks (fail early)
  stopifnot(
    "TIMESTAMP" %in% names(alldat),
    "SN" %in% names(alldat),
    "Date" %in% names(metadat),
    "Time_start"%in% names(metadat),

```

```

  "Obs_length"%in% names(metadat),
  "Plot"      %in% names(metadat)
)

stopifnot(all(metadat$Obs_length > 0, na.rm = TRUE))
stopifnot(anyDuplicated(metadat$Plot) == 0)
# Instruments present in BOTH datasets
instruments <- intersect(
  unique(alldat$SN),
  unique(metadat$SN)
)
message("Instruments to process: ", paste(instruments, collapse = ", "))
results <- lapply(instruments, function(sn) {
  message("Processing SN: ", sn)
  dat_i <- alldat %>%
    filter(SN == sn)
  meta_i <- metadat %>%
    filter(SN == sn) %>%
    arrange(Date, Time_start) %>%
    mutate(metadat_row = row_number())
  stopifnot(nrow(meta_i) > 0)
# Actual matching
dat_i$metadat_row <- ffi_metadata_match(
  data_timestamps = dat_i$TIMESTAMP,
  start_dates     = meta_i$Date,
  start_times     = meta_i$Time_start,
  obs_lengths     = meta_i$Obs_length )
# Attach metadata (vectorized, fast)
dat_i <- dat_i %>%
  mutate(
    Plot      = meta_i$Plot[metadat_row],
    Volume    = meta_i$Volume[metadat_row],
    Temp      = meta_i$Temp[metadat_row],
    Area      = meta_i$Area[metadat_row],
    Dead_band = meta_i$Dead_band[metadat_row]
  )
# Drop unmatched rows
n_before <- nrow(dat_i)
dat_i <- dat_i %>% filter(!is.na(metadat_row))
n_after <- nrow(dat_i)
message("  Matched rows: ", n_after, " / ", n_before)
dat_i })
alldat_matched <- rbindlist(results, fill = TRUE)

```

```

#Final sanity checks
stopifnot(
  !anyNA(alldat_matched$Plot),
  nrow(alldat_matched) > 0
)
message("Matching finished successfully")
return(alldat_matched)}

#running the function
message("Running final metadata matching...")
alldat_matched <- match_flux_metadata(alldat, metadat)
message("Matched dataset rows: ", nrow(alldat_matched))
head(alldat_matched)

#saving matched data and metadata as raw data is too big (today, Licor files leave in the CO2
saveRDS(alldat_matched, file = "alldat_matched_2023.rds")

#can remove any row that have NA's in the CH4 column
#alldat <- alldat %>%
# filter(!is.na(CH4))

```

6 Reading matched data and metadata

```

# after you match the data for the first time - we can just read the .rds file to save time
alldat_matched <- readRDS("alldat_matched_2023.rds")

```

```

##Extracting initial CO2 concentration

```

```

## Extract initial CH4 concentration (t0) for each Plot for QAQC later
#saving df to use after flux calculation
df_CH4_in <- alldat_matched %>%
  arrange(Plot, TIMESTAMP) %>%
  group_by(Plot) %>%
  summarise(CH4_initial = first(CH4), .groups = "drop")

```

```

##Creating temperature csv from the L2-weather station in the wetland

```

```

# Getting Air temp from the weather station
## Read L2 temperature data from ZIP stored outside GitHub,
## extract desired variable, save lightweight CSV,
## and remove temporary unzipped files afterwards
## Read extracted L2 Parquet files and save temperature time series
BASE_DIR <- "D:/COMPASS_L2-v2-1/v2-1"
SITE <- "(OWC|CRC|PTR)"
VARIABLE <- "wx-tempavg15"
OUTFILE <- "Raw Data/L2_temp_timeseries.csv"
# Build regex that matches REAL L2 filenames
regex <- paste0(SITE, ".*", VARIABLE, ".*_L2_v2-1.*\\.parquet$")
# Locate Parquet files
files <- list.files(BASE_DIR, pattern = regex, recursive = TRUE, full.names = TRUE)
message("Files found: ", length(files))
print(basename(files))
# Fail fast if nothing is found
stopifnot(length(files) > 0)
# Read and combine Parquet files
dat <- lapply(files, function(f) {
  message("Reading ", basename(f))
  read_parquet(f)
})

dat <- bind_rows(dat)
# Optional QA filtering (only if column exists)
if ("QA_flag" %in% names(dat)) {
  dat <- dat %>% filter(QA_flag == 0)
}
# Keep only necessary columns
temps <- dat %>%
  select(TIMESTAMP, Plot, Site, Value) %>%
  rename(Temp = Value)
head(temps)
# Save lightweight output only
write.csv(
  temps,
  file = OUTFILE,
  row.names = FALSE
)

```

7 Reading Temp df to merge to metadat

```
# reading temp
temps <- read.csv("Raw Data/L2_temp_timeseries.csv")
# make sure timestamps are in the correct format
# convert to tibble early to avoid data.table list-column issues
alldat_matched <- as_tibble(alldat_matched) %>%
mutate(across(where(is.list), ~ as.numeric(unlist(.))))
alldat_matched$TIMESTAMP <- as.POSIXct(alldat_matched$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")
alldat_matched$Dead_band <- as.numeric(alldat_matched$Dead_band)
# fix midnight timestamps in L2 data
temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)] <-
paste0(temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)], " 00:00:00")
temps$TIMESTAMP <- as.POSIXct(temps$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")

# time rounding
alldat_matched$TIMESTAMP_round <- round_date(alldat_matched$TIMESTAMP, "60 mins")
temps$TIMESTAMP_round <- round_date(temps$TIMESTAMP, "60 mins")
# one observation per hour
temps_clean <- temps %>%
group_by(TIMESTAMP_round, Site) %>%
summarize(Temp = mean(Temp), .groups = "drop")
# join weather station temp
alldat_matched <- alldat_matched %>%
left_join(temps_clean, by = c("Site", "TIMESTAMP_round"))
# DIAGNOSTIC: which plots are missing Temp after join
temp_missing_check <- alldat_matched %>%
group_by(Plot, Site) %>%
summarise(
n_obs = n(),
n_temp_na = sum(is.na(Temp)),
pct_temp_na = round(100 * n_temp_na / n_obs, 1),
month = month(first(TIMESTAMP)),
.groups = "drop"
) %>%
filter(n_temp_na > 0)
message("Plots with missing Temp: ", nrow(temp_missing_check))
```

Plots with missing Temp: 2

```
print(temp_missing_check)
```

```
# A tibble: 2 x 6
```

	Plot <chr>	Site <chr>	n_obs <int>	n_temp_na <int>	pct_temp_na <dbl>	month <int>
1	2/21/2023_CRC_UP_304_68	CRC	720	720	100	2
2	2/21/2023_CRC_UP_305_69	CRC	360	360	100	2

```
# IMPUTATION: monthly mean per site as fallback
monthly_temp <- alldat_matched %>%
mutate(month = month(TIMESTAMP)) %>%
  filter(!is.na(Temp)) %>%
group_by(Site, month) %>%
summarise(Temp_monthly_mean = mean(Temp), .groups = "drop")
alldat_matched <- alldat_matched %>%
mutate(
month = month(TIMESTAMP),
Temp = as.numeric(Temp)
) %>%
left_join(monthly_temp, by = c("Site", "month")) %>%
mutate(
Temp_source = case_when(
!is.na(Temp) ~ "weather_station",
!is.na(Temp_monthly_mean) ~ "monthly_mean_imputed",
TRUE ~ "still_missing"
),
Temp = coalesce(Temp, Temp_monthly_mean)
) %>%
select(-Temp_monthly_mean, -month)
# FINAL CHECK
message("Temp source summary:")
```

Temp source summary:

```
alldat_matched$Temp_source <- as.character(alldat_matched$Temp_source)
table(alldat_matched$Temp_source)
```

monthly_mean_imputed	weather_station
1080	131125

```

still_missing <- alldat_matched %>%
filter(Temp_source == "still_missing") %>%
distinct(Plot, Site)
if (nrow(still_missing) > 0) {
message("WARNING: ", nrow(still_missing), " plots still have no Temp after imputation:")
print(still_missing)
} else {
  message("All plots have Temp after imputation.")
}

```

All plots have Temp after imputation.

8 fixing some Deadbands and stop time

```

# Table of plots with custom deadband adjustments
time_overrides <- tibble::tibble(
  Plot = c("7/13/2023_CRC_UP_302_226", "5/10/2023_OWC_TR_121_163", "7/13/2023_CRC_TR_315_237",
    "3/28/2023_OWC_TR_127_131", "6/29/2023_CRC_UP_305_213", "5/10/2023_OWC_TR_127_168",
    "6/29/2023_CRC_UP_306_214", "7/13/2023_CRC_TR_314_236", "3/24/2023_CRC_UP_2_103", "2/
    "6/29/2023_CRC_TR_311_219", "2/10/2023_OWC_UP_101_49", "7/13/2023_CRC_UP_307_B_241",
    "7/13/2023_CRC_UP_308_B_240", "2/21/2023_CRC_UP_308_72", "2/21/2023_CRC_TR_314_78",
    "2/21/2023_CRC_UP_304_68"
  ),
  dead_band_manual = c(20, 11, 10, 20, 20, 10, 20, 20, 10, 23, 60, 60, 6, 14, 30, 25, 15, 5, 10, 10, 10
  ),
  stop_offset_secs = NA_real_ )

#apply new incubation period
alldat_trimmed <- alldat_matched %>%
left_join(time_overrides, by = "Plot") %>%
group_by(Plot) %>%
mutate(
  t_start_original = min(TIMESTAMP),
  t_end_original = max(TIMESTAMP),
  # deadband manual or metadat
  t_start_adjusted = if_else(
    !is.na(dead_band_manual),
    t_start_original + dead_band_manual,
    t_start_original + as.numeric(coalesce(first(Dead_band), 0))
  )
)

```

```

),
# new offset
t_end_adjusted = if_else(!is.na(stop_offset_secs),
t_end_original + stop_offset_secs,
t_end_original
)
) %>%
filter(TIMESTAMP >= t_start_adjusted,
TIMESTAMP <= t_end_adjusted) %>%
ungroup() %>%
select(-t_start_original, -t_end_original,
-t_start_adjusted, -t_end_adjusted,
-dead_band_manual, -stop_offset_secs)

# alldat_trimmed %>%
# filter(Plot == "8/17/2022_OWC_UP_6_147") %>%
# ggplot(aes(TIMESTAMP, CO2)) +
# geom_point() + theme_classic()

```

9 Calculate fluxes

```

# TO Do - NEED TO PULL TEMP from the weather station
#change the units
alldat_trimmed$CH4_nmol <- ffi_ppb_to_nmol(alldat_trimmed$CH4,
                                           volume = alldat_trimmed$Volume, # m3
                                           temp = alldat_trimmed$Temp)      # degrees C

```

Assuming atm = 101325 Pa

Using R = 8.31446261815324 m3 Pa K-1 mol-1

```

#> Assuming atm = 101325 Pa
#> Using R = 8.31446261815324 m3 Pa K-1 mol-1

alldat_trimmed$CH4_nmol_m2 <- alldat_trimmed$CH4_nmol / 0.16

fluxes <- ffi_compute_fluxes(alldat_trimmed ,

```


[illegible]

[illegible]

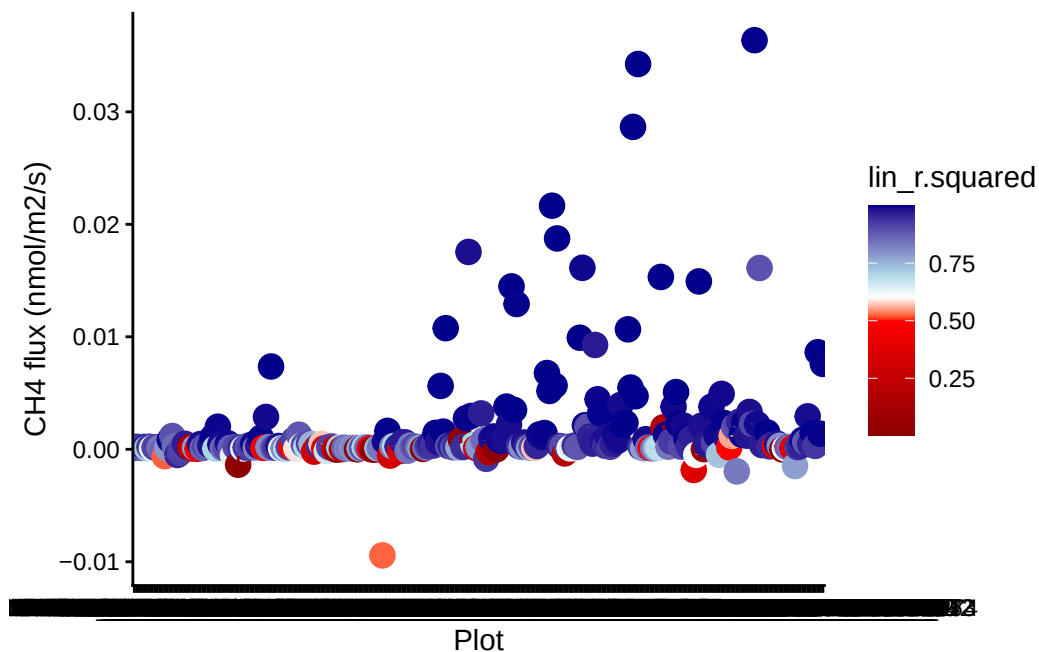
[illegible]

```
head(fluxes)
```

	Plot	TIMESTAMP	HM81_AIC	HM81_flux.estimate		
1	1/12/2023_PTR_TR_1_26	2023-01-12 11:25:29	-4314.439	0.0001969688		
2	1/12/2023_PTR_TR_2_27	2023-01-12 11:41:49	NA	NA		
3	1/12/2023_PTR_TR_3_28	2023-01-12 11:55:29	-4029.374	0.0004118174		
4	1/12/2023_PTR_TR_4_29	2023-01-12 12:07:29	-4188.030	0.0001144522		
5	1/12/2023_PTR_TR_4_B_30	2023-01-12 12:18:29	-3637.840	0.0001978180		
6	1/12/2023_PTR_TR_5_31	2023-01-12 12:26:29	-4545.038	0.0003195774		
	HM81_p.value	HM81_r.squared	HM81_RMSE	lin_AIC	lin_flux.estimate	
1	1.385494e-133	0.6366686	0.006620163	-4796.868	1.158595e-04	
2	NA	NA	NA	-6065.952	2.168277e-04	
3	8.679474e-139	0.6509263	0.008395318	-4892.484	1.568739e-04	
4	9.674015e-106	0.5498878	0.007355592	-4920.631	1.176997e-04	
5	4.363709e-127	0.6180428	0.011634247	-4997.471	2.126669e-04	
6	5.757338e-146	0.6696836	0.005462762	-4648.933	9.311583e-05	
	lin_flux.std.error	lin_int.estimate	lin_int.std.error	lin_p.value		
1	2.087701e-06	80.26672	0.0003606962	4.716390e-238		
2	1.475225e-06	80.40066	0.0003230153	0.000000e+00		
3	1.927807e-06	80.62782	0.0003330711	0.000000e+00		
4	1.883115e-06	80.96505	0.0003253495	2.163633e-264		
5	1.766312e-06	81.11470	0.0003051693	0.000000e+00		
6	2.361606e-06	81.25003	0.0004080195	1.813546e-168		
	lin_r.squared	lin_RMSE	poly_AIC	poly_r.squared	poly_RMSE	rob_AIC
1	0.8374040	0.004428657	-4836.566	0.8488251	0.004277446	-4796.607
2	0.9661016	0.004461250	-6233.810	0.9729621	0.003989579	-6065.534
3	0.9171719	0.004089474	-4900.911	0.9188697	0.004054128	-4892.429
4	0.8672463	0.003994667	-4923.116	0.8686735	0.003979797	-4920.496
5	0.9603831	0.003746893	-5003.421	0.9610346	0.003722183	-4997.332
6	0.7222026	0.005009696	-4982.261	0.8416703	0.003788400	-4648.405
	rob_converged	rob_flux.estimate	rob_flux.std.error	rob_RMSE		
1	TRUE	1.158814e-04	2.135347e-06	0.004525445		
2	TRUE	2.160745e-04	1.553220e-06	0.004399491		
3	TRUE	1.566446e-04	1.962668e-06	0.004430106		
4	TRUE	1.179333e-04	1.882214e-06	0.003824707		
5	TRUE	2.125872e-04	1.855980e-06	0.003879984		
6	TRUE	9.217551e-05	2.520499e-06	0.004703436		
	TIMESTAMP_min	TIMESTAMP_max				
1	2023-01-12 11:23:00	2023-01-12 11:27:59				
2	2023-01-12 11:38:40	2023-01-12 11:44:59				
3	2023-01-12 11:53:00	2023-01-12 11:57:59				
4	2023-01-12 12:05:00	2023-01-12 12:09:59				

```
5 2023-01-12 12:16:00 2023-01-12 12:20:59
6 2023-01-12 12:24:00 2023-01-12 12:28:59
```

```
#take a look at the fluxes we calculated with R2
ggplot(fluxes, aes(Plot, lin_flux.estimate, color = lin_r.squared)) +
  geom_point(size=4) + theme_classic() +
  geom_linerange(aes(ymin = lin_flux.estimate- lin_flux.std.error,
                    ymax = lin_flux.estimate+ lin_flux.std.error)) +
  scale_color_gradientn(colours = c("darkred", "red", "white", "lightblue", "darkblue"),
                       values = c(0, 0.5, 0.6, 0.7, 1)) +
  ylab("CH4 flux (nmol/m2/s)")
```



```
#sort fluxes by R2
fluxes_good <- as.data.frame(subset(fluxes, lin_r.squared >= .90, select = Plot:TIMESTAMP_ma
fluxes_lowr2 <- as.data.frame(subset(fluxes, lin_r.squared < .90 & lin_flux.estimate < 1, se
fluxes_weird <- as.data.frame(subset(fluxes, lin_r.squared < .90, select = Plot:TIMESTAMP_ma

#find fluxes that represent the first and fourth quartile of the data
low <- quantile(fluxes$lin_flux.estimate, 0.05)
high <- quantile(fluxes$lin_flux.estimate, 0.95)

fluxes_low <- as.data.frame(subset(fluxes, lin_flux.estimate <= low, select = Plot:TIMESTAMP_
```

```
fluxes_high <- as.data.frame(subset(fluxes, lin_flux.estimate >= high, select = Plot:TIMESTAMP))
tails <- rbind(fluxes_low, fluxes_high)
```

10 check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)

```
#### If something is in a V shape should we take the second part to make a positive linear c

# grab trim limits for each overridden Plot
trim_limits <- alldat_trimmed %>%
  filter(Plot %in% time_overrides$Plot) %>%
  group_by(Plot) %>%
  summarise(
    t_start_trim = min(TIMESTAMP),
    t_end_trim = max(TIMESTAMP) )
# use full alldat_matched + trim limits to classify each point
r_all <- alldat_matched %>%
  filter(Plot %in% time_overrides$Plot) %>%
  left_join(trim_limits, by = "Plot") %>%
  group_by(Plot) %>%
  mutate(
    time_sec = as.numeric(difftime(TIMESTAMP, min(TIMESTAMP), units = "secs")),
    point_status = case_when(
      TIMESTAMP < t_start_trim ~ "Discarded (deadband)",
      TIMESTAMP > t_end_trim ~ "Discarded (stop trim)",
      TRUE ~ "Used" )) %>%
  ungroup()
# plot all overridden plots
ggplot(r_all, aes(x = TIMESTAMP, y = CO2, color = point_status)) +
  geom_point(size = 1.8) +
  facet_wrap(~ Plot, scales = "free", ncol = 2) +
  scale_color_manual(
    values = c(
      "Used" = "#E41A1C",
      "Discarded (deadband)" = "grey60",
      "Discarded (stop trim)" = "grey20"
    )
  ) +
```

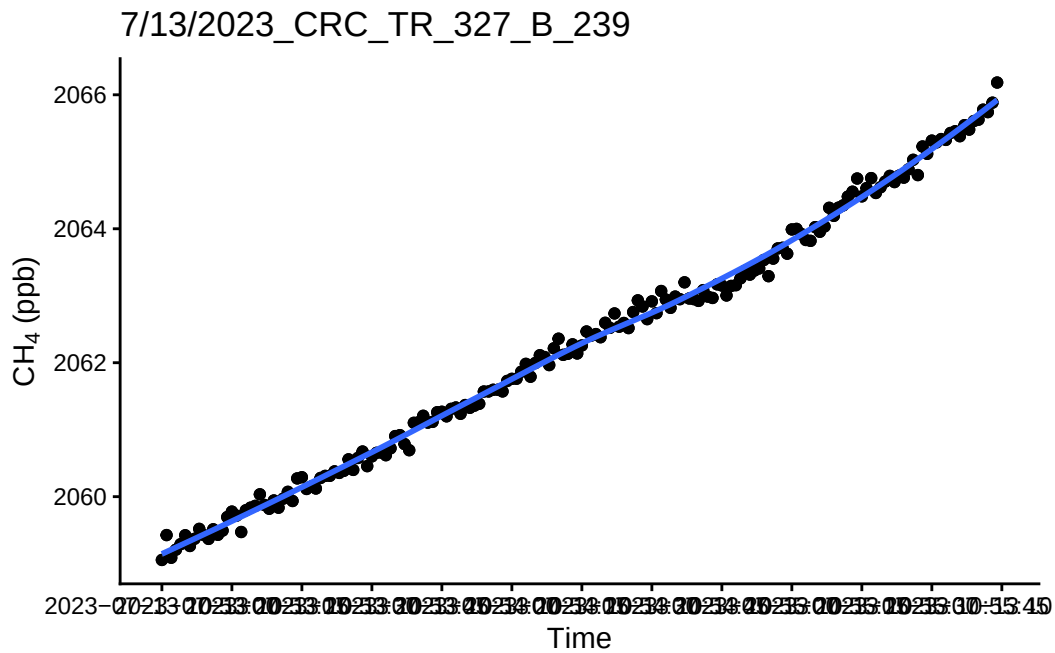
```

theme_classic() +
ylab(expression(paste(CO[2], " (ppm)"))) +
xlab("Time") +
labs(
title = "Incubation Period Adjustments",
color = "Point Status") +
  theme(legend.position = "bottom")

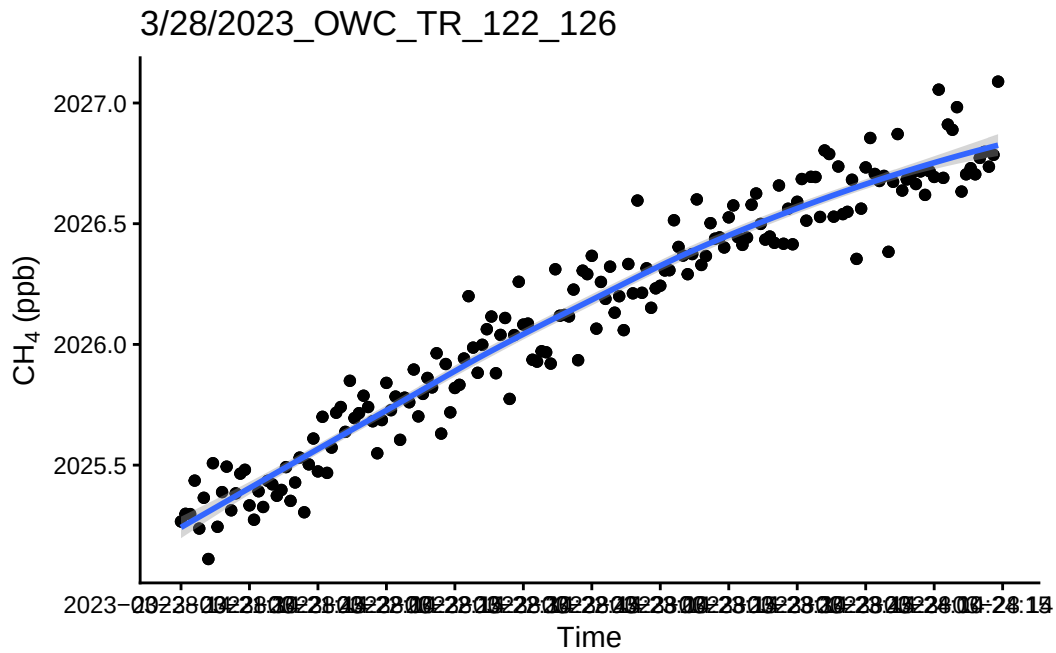
```

11 Spot check a few fluxes with $R^2 > 0.9$

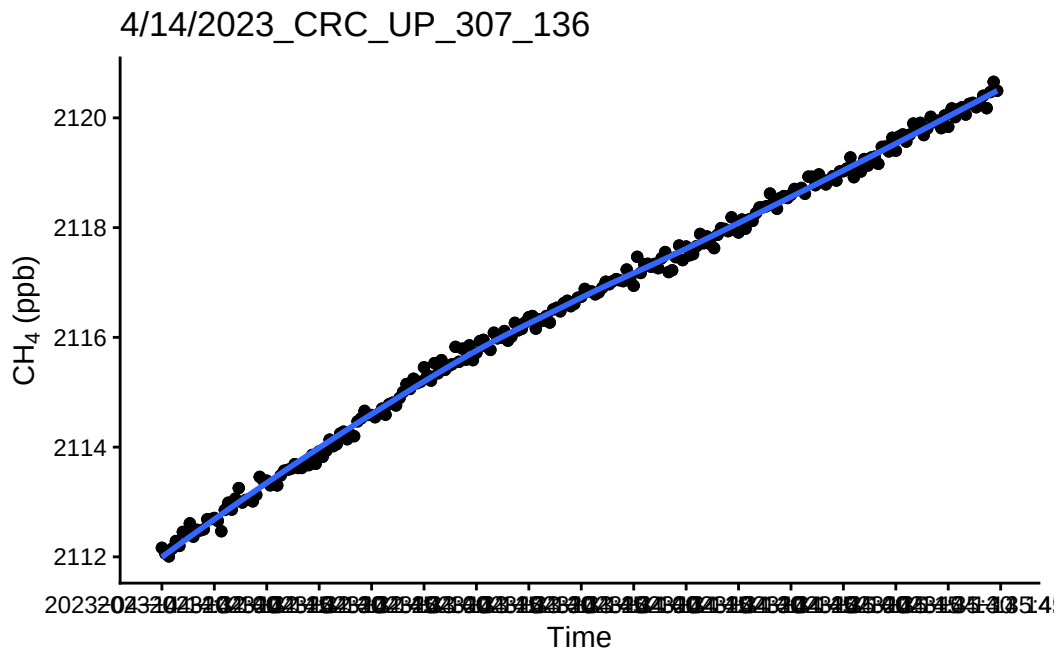
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



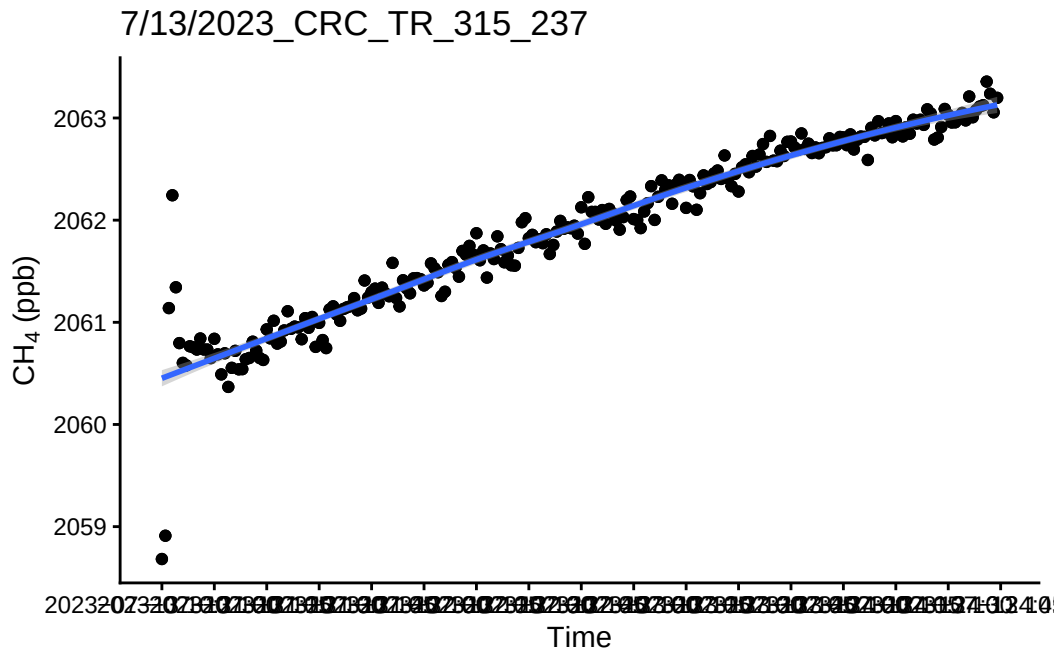
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



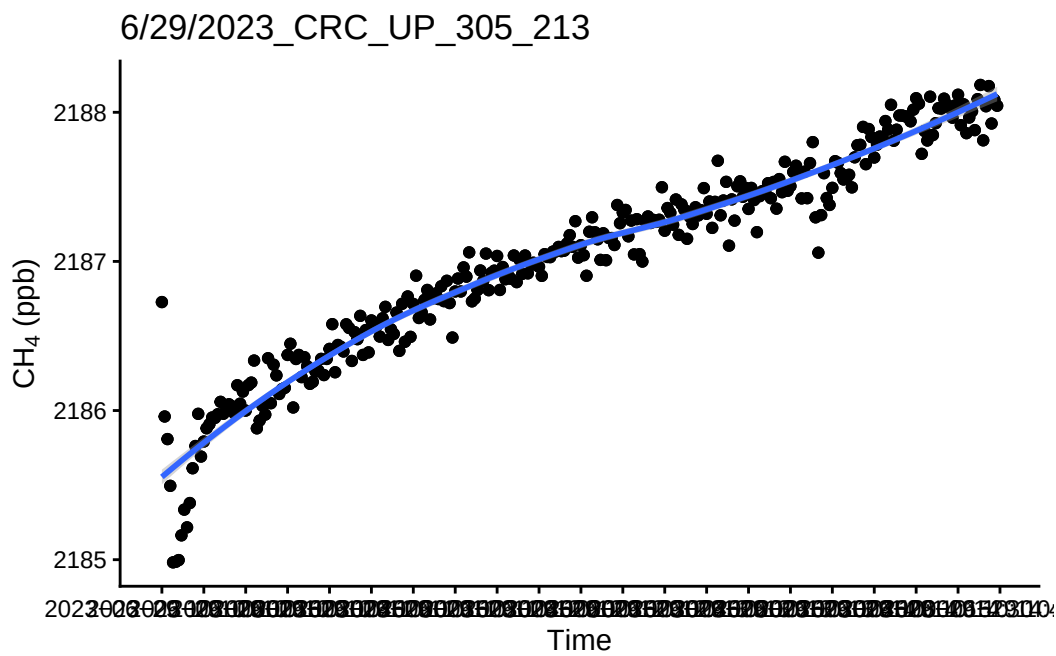
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

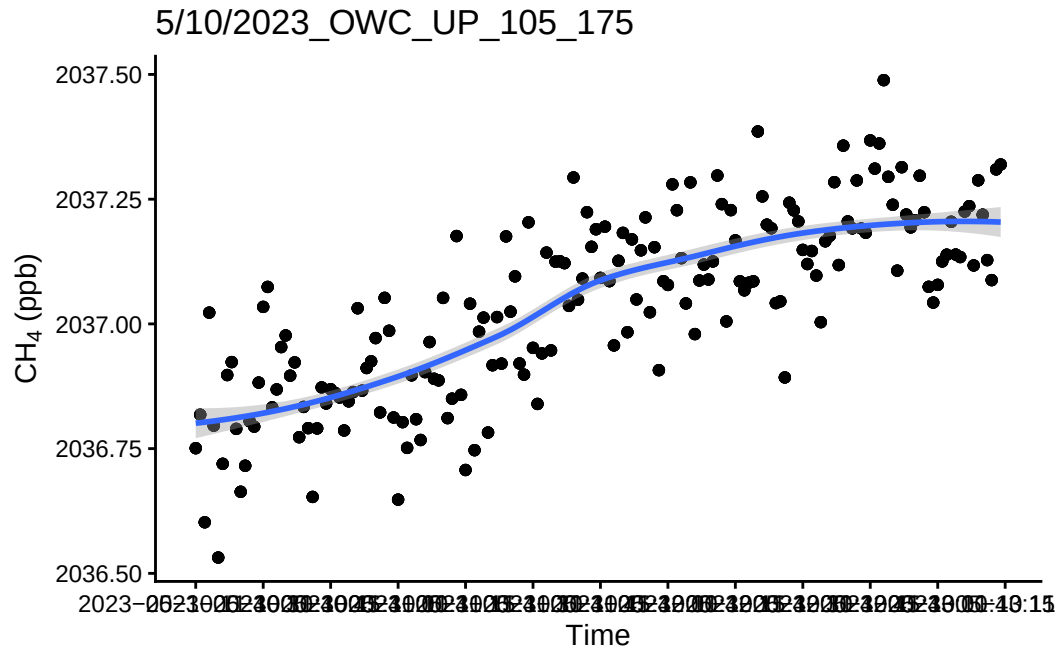


``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

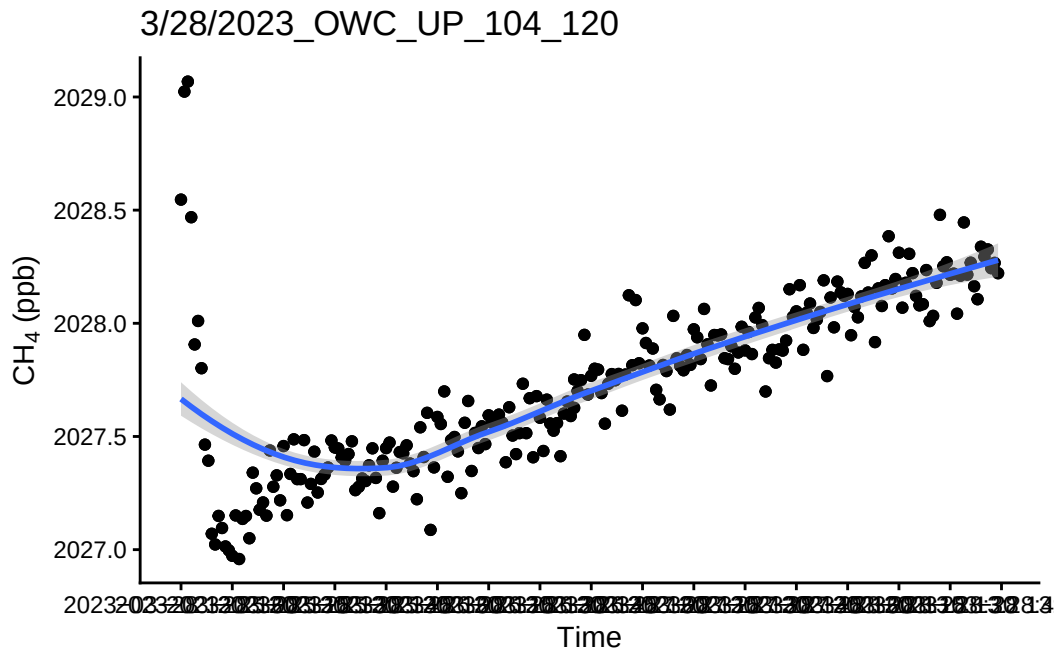


12 Take a look at the fluxes <0.9 and a flux estimate below 1

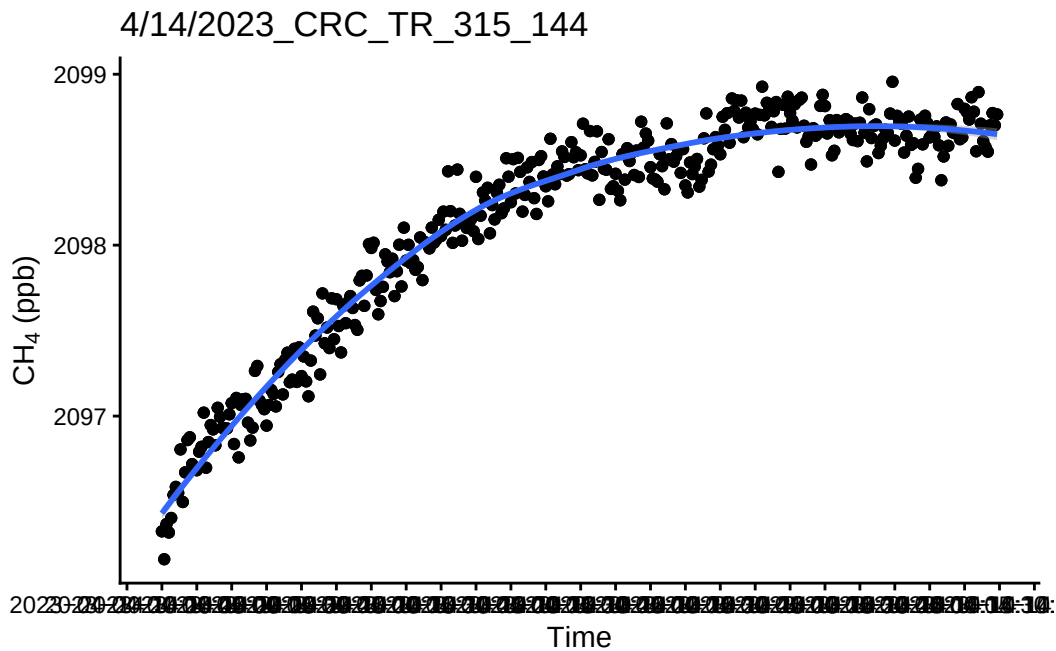
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



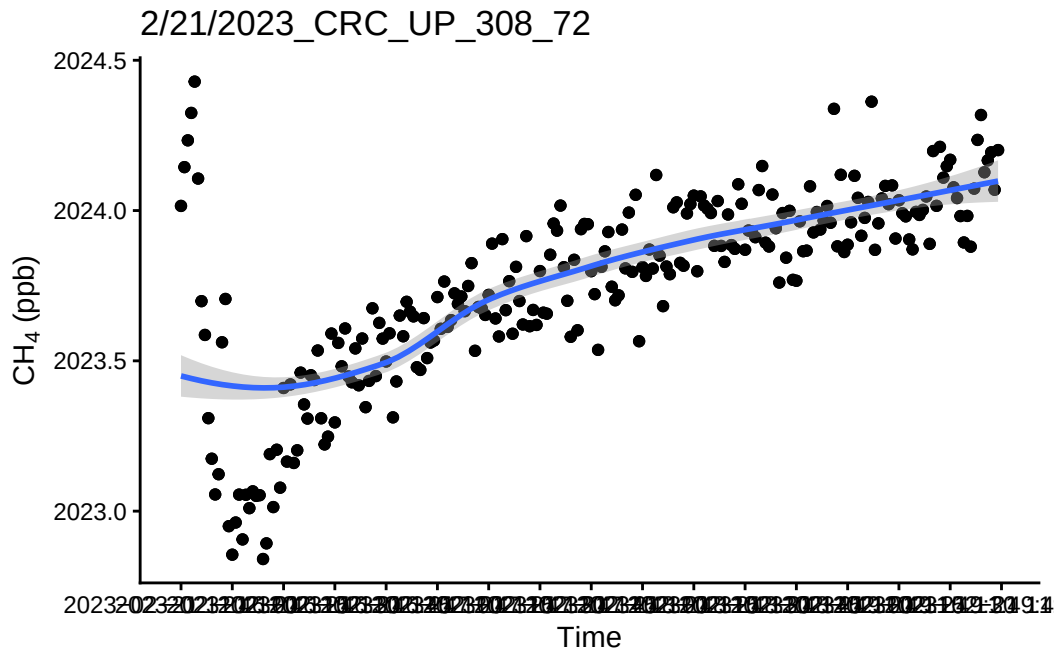
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



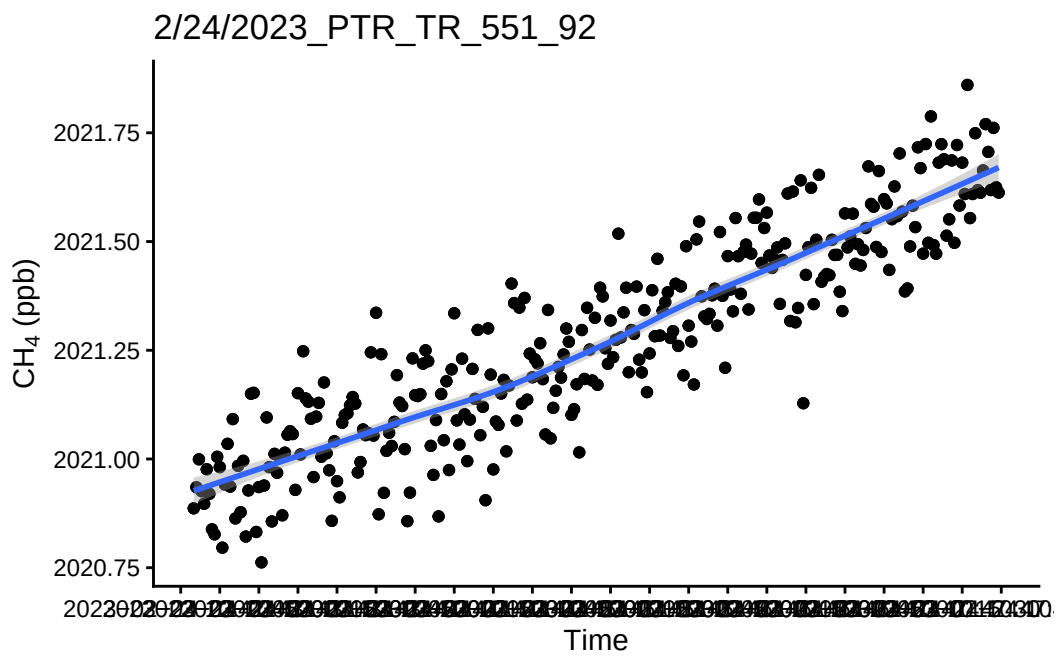
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



13 Take a look at the bad fluxes < 0.9 and have a high flux estimate

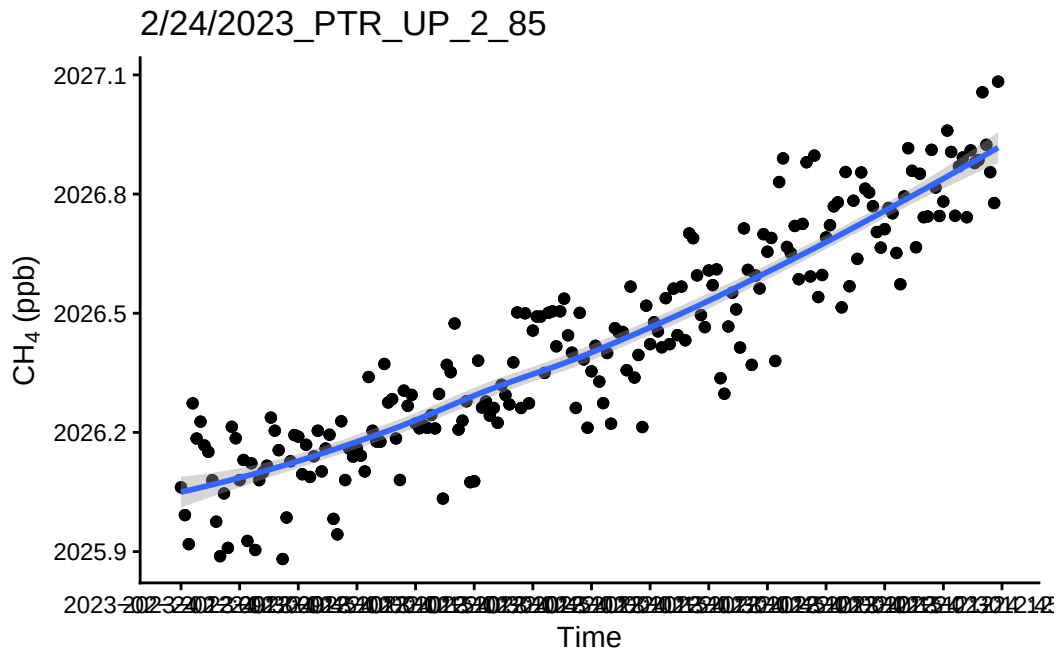
14 See if we want to remove these fluxes or if we want to try and recalculate these ones?

```
### pull out fluxes with R2 < 0.9 and put them in a list
bad_plots <- unique(fluxes_weird$Plot)

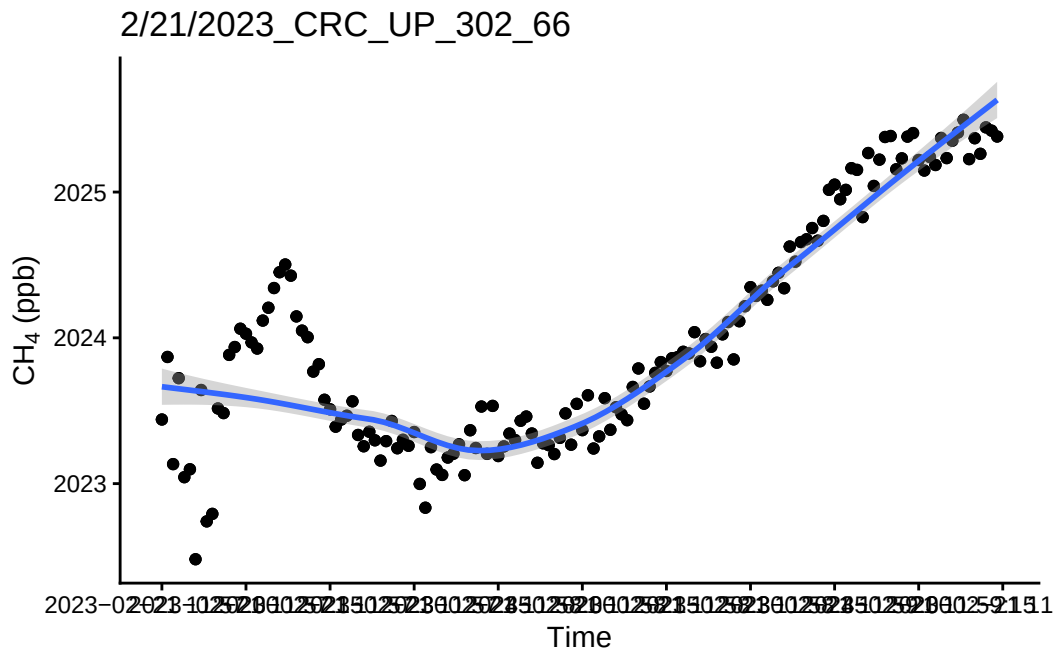
# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
bad_sample <- sample(bad_plots, 5, replace=FALSE)

#Loop to run five random plots subset above
for(P in bad_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CH [4], " (ppb)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
  print(p1)
}
```

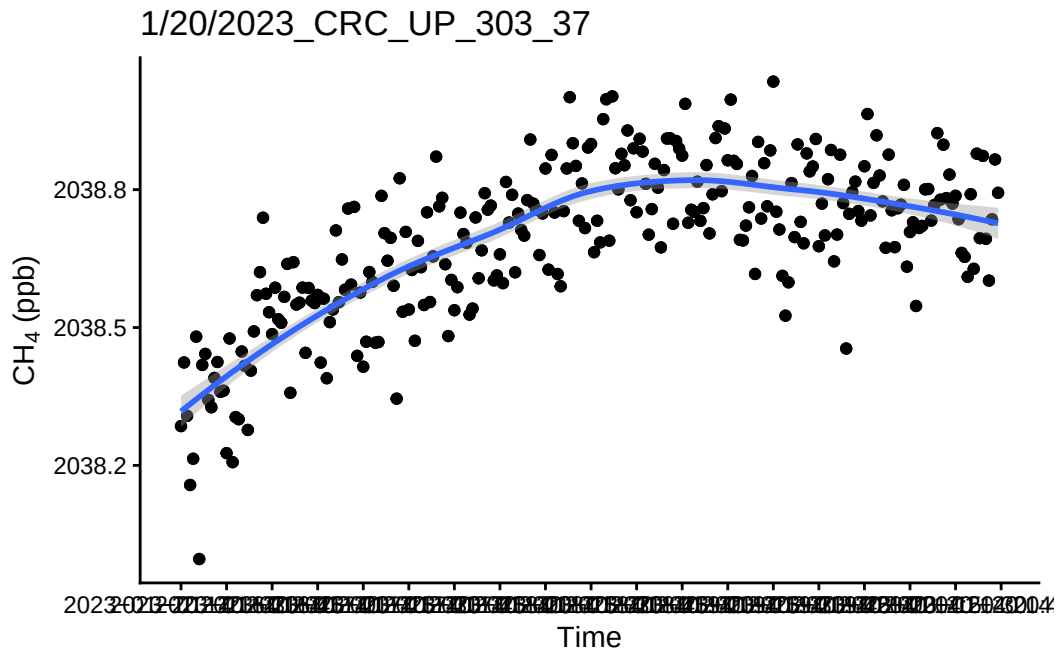
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



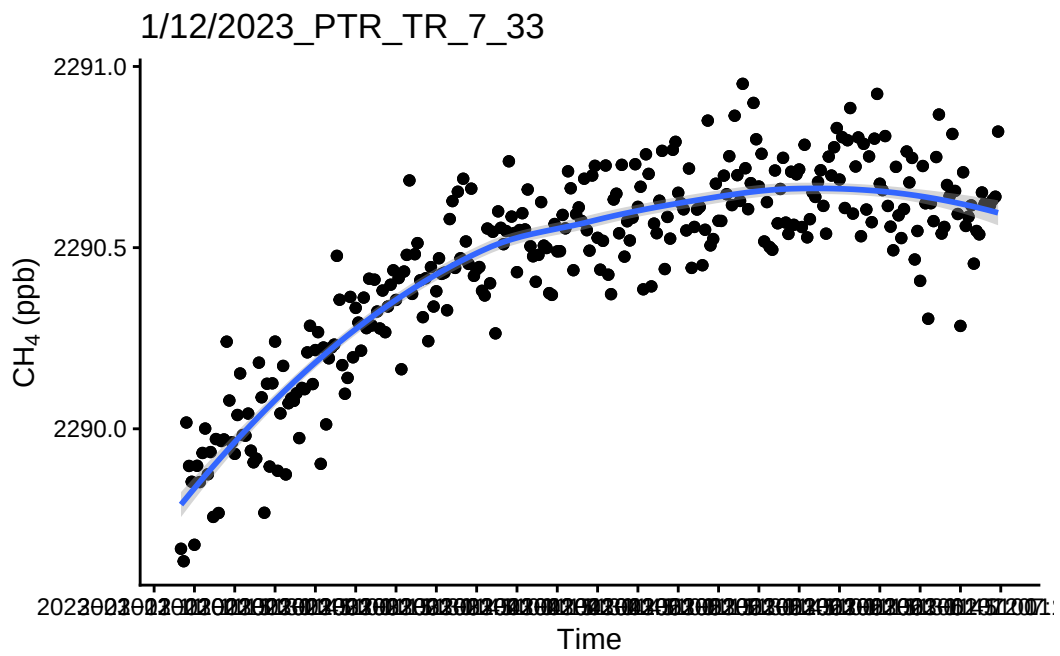
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



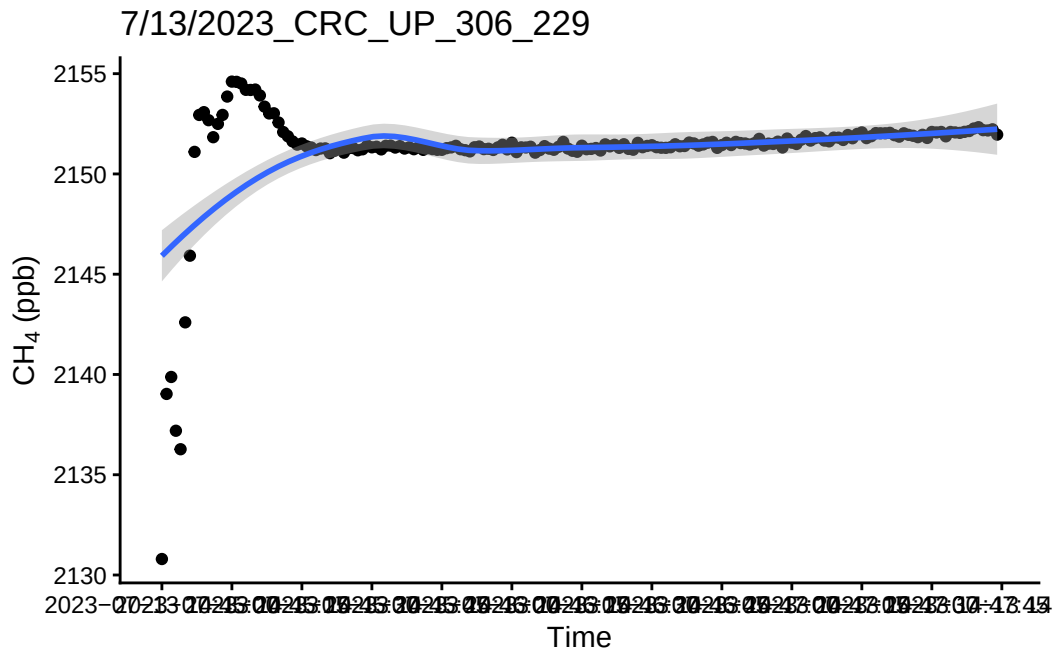
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



#####

15 Look at the really high or really low ones to see if there is a timing issue or something

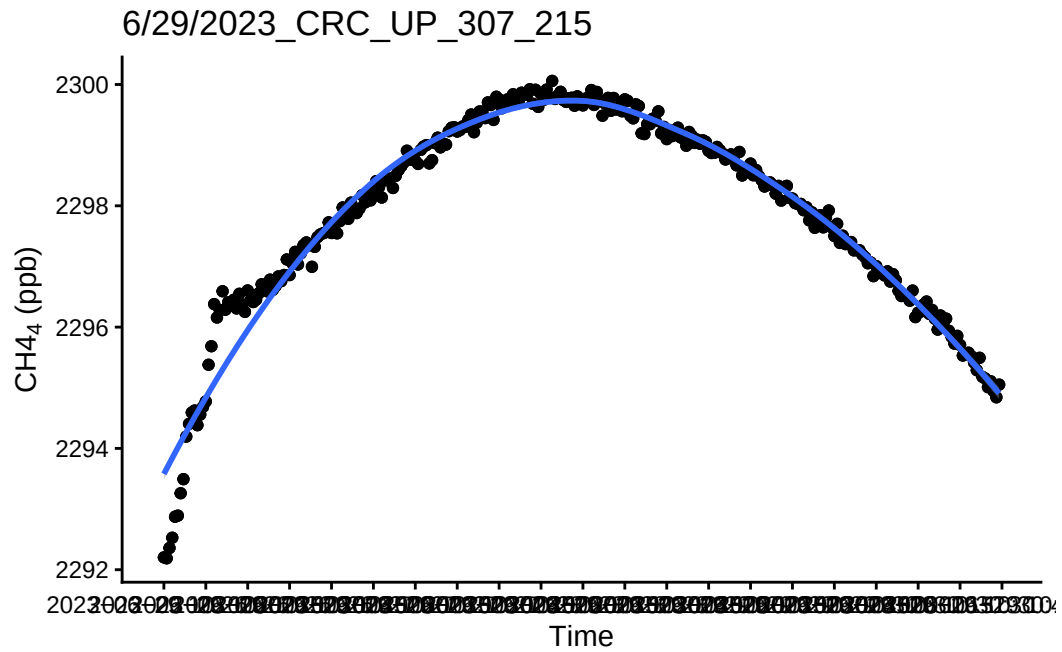
```
### pull out fluxes in the 5% and 95% tails to look at really high and really low
tail_plots <- unique(tails$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
tail_sample <- sample(tail_plots, 2, replace=FALSE)

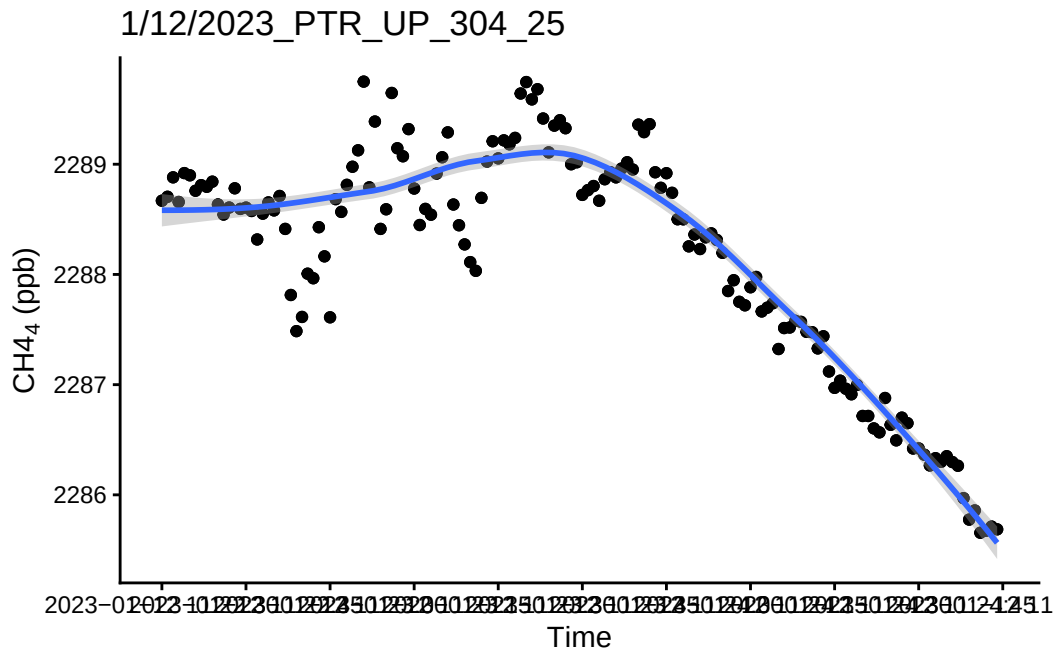
#Loop to run five random plots subset above
for(P in tail_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadata_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CH4 [4], " (ppb)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
```

```
print(p1)
}
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



```
#would be nice to put on the plot the flux estimate and if it is a high or low or something
#####
```

16 Check all fluxes

```
##Put all fluxes together
fluxes_all <- as.data.frame(subset(fluxes, select = Plot:TIMESTAMP_max))

all_plots <- unique(fluxes_all$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
all_sample <- sample(all_plots, 5, replace=FALSE)

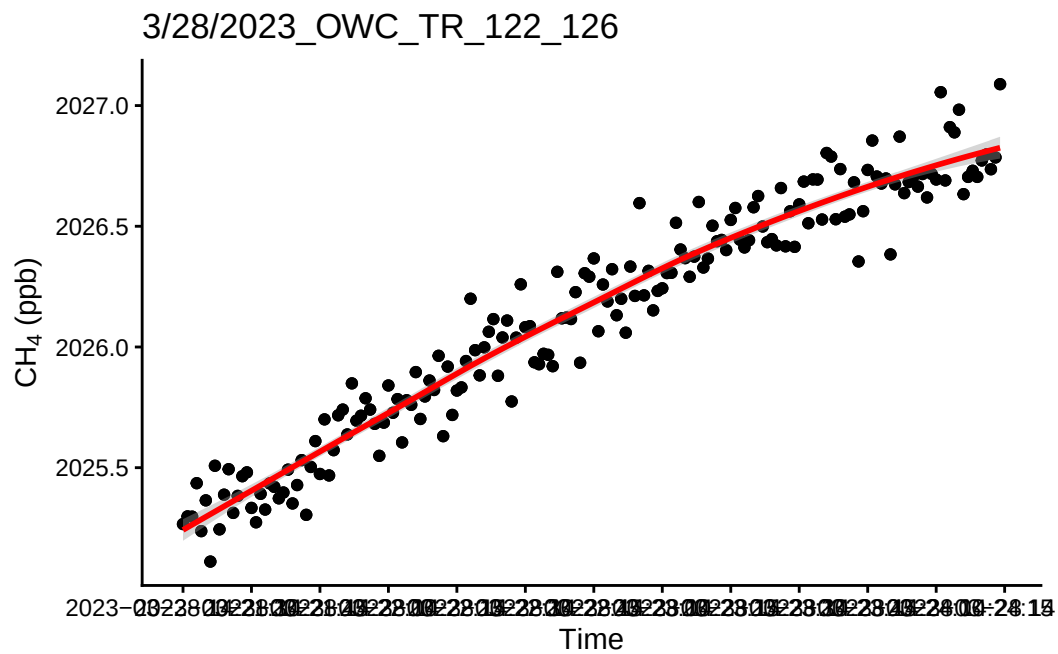
#Loop to run five random plots subset above
for(P in all_sample){
  r1 <- subset(alldat_matched, Plot== P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CH4)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CH [4], " (ppb)"))) +
```

```

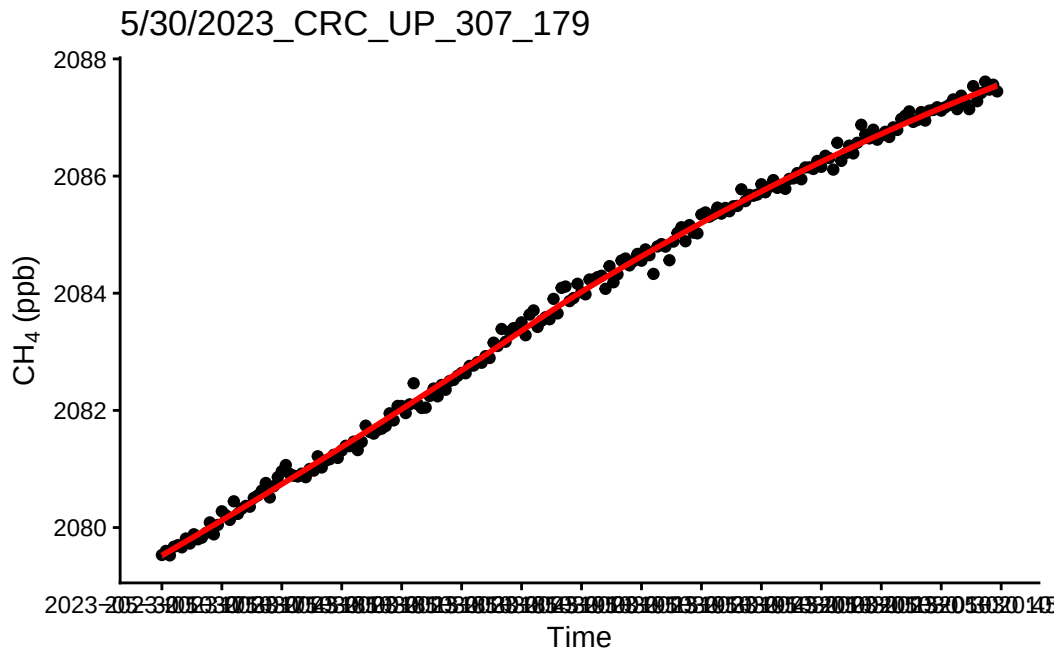
xlab("Time") + labs(title = P) +
geom_smooth(color="red")
print(p1)
}

```

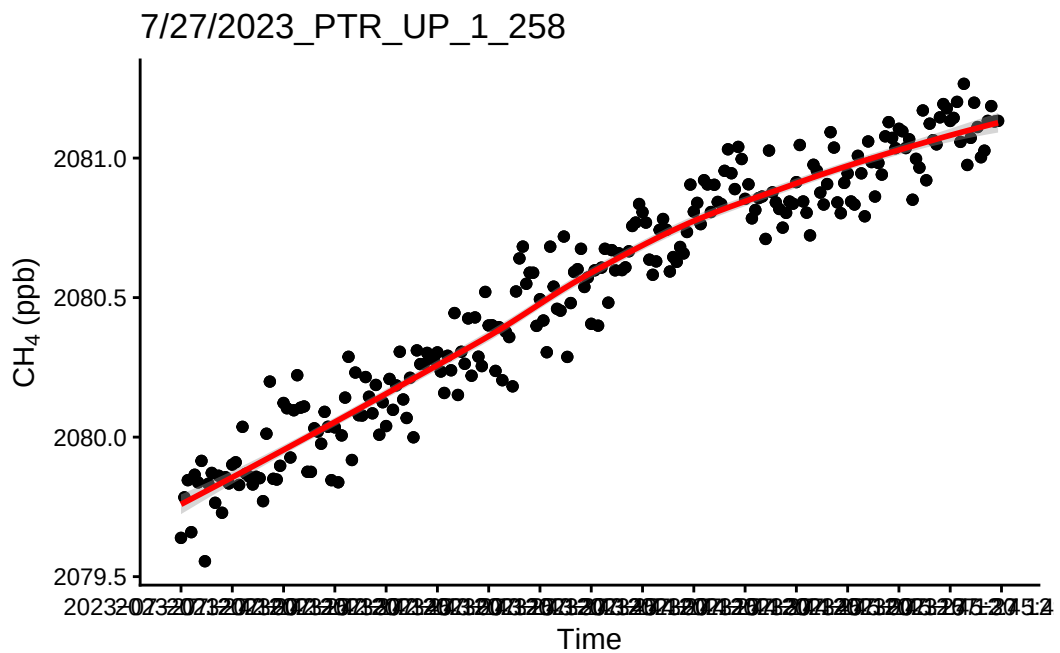
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



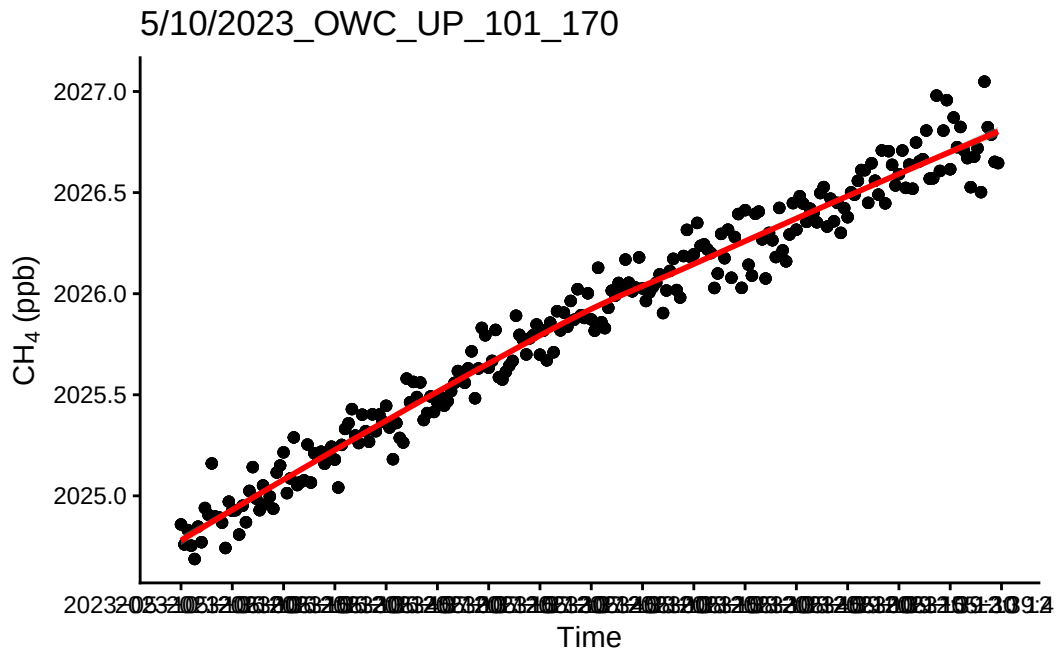
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



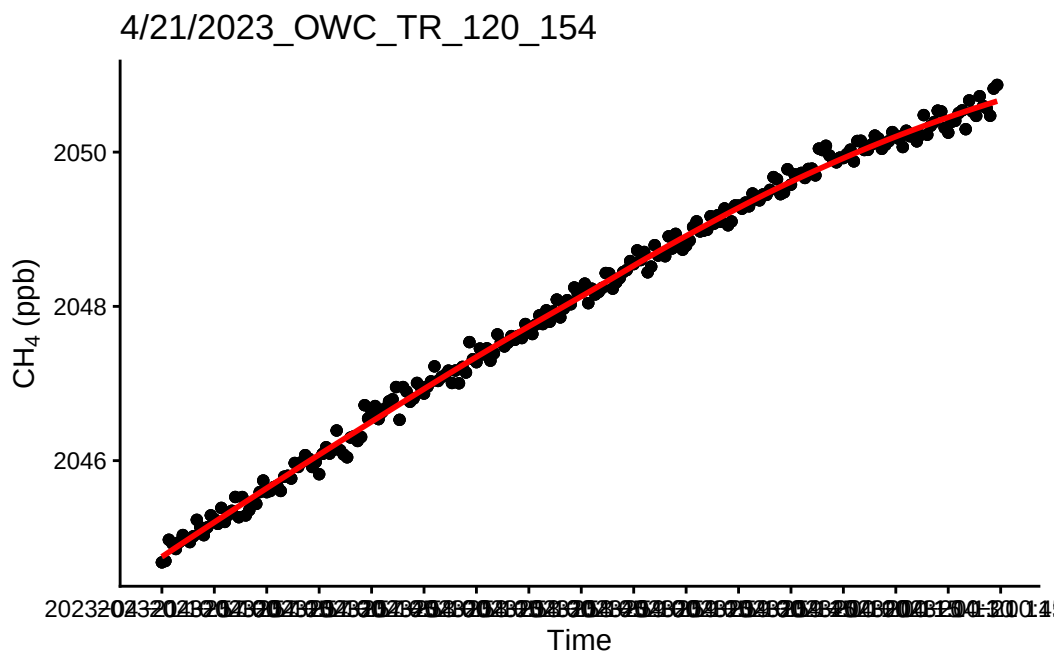
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



```
#####
```

17 QAQC - Flags

```
# QAQC flags
fluxes_flagged <- fluxes %>%
mutate(
# model fit flags
CH4_lin_pass = (lin_r.squared >= 0.85 | lin_RMSE < 5) & !is.na(lin_r.squared),
CH4_HM81_pass = (HM81_r.squared >= 0.85 | HM81_RMSE < 5) & !is.na(HM81_r.squared),
CH4_poly_pass = (poly_r.squared >= 0.85 | poly_RMSE < 5) & !is.na(poly_r.squared),
# negative flux flag (CH4 negative fluxes may be valid, but flagging anyway)
CH4_neg_flux = lin_flux.estimate < 0 | is.na(lin_flux.estimate),
# rejection flags
CH4_rejected_model = !(CH4_lin_pass | CH4_HM81_pass | CH4_poly_pass)
)
r2_summary <- fluxes_flagged %>%
summarise(
CH4_total_obs = n(),
CH4_n_rejected_model = sum(CH4_rejected_model, na.rm = TRUE),
CH4_pct_rejected_total = 100 * CH4_n_rejected_model / CH4_total_obs,
CH4_n_lin_pass = sum(CH4_lin_pass, na.rm = TRUE),
CH4_n_HM81_pass = sum(CH4_HM81_pass, na.rm = TRUE),
CH4_n_poly_pass = sum(CH4_poly_pass, na.rm = TRUE)
)
# formatted table for PDF
r2_summary %>%
tidyr::pivot_longer(everything(), names_to = "Metric", values_to = "Value") %>%
mutate(
Metric = recode(Metric,
CH4_total_obs = "Total observations",
CH4_n_rejected_model = "Rejected - poor model fit (n)",
CH4_pct_rejected_total = "Rejected - total (%)",
CH4_n_lin_pass = "Passed - Linear model (n)",
CH4_n_HM81_pass = "Passed - HM81 model (n)",
CH4_n_poly_pass = "Passed - Polynomial model (n)"
),
Value = ifelse(Metric == "Rejected - total (%)",
round(Value, 1),
as.integer(Value))
```

```

    ) %>%
knitr::kable(
  caption = "QA/QC CH4 Flux Summary",
  align = c("l", "r"),
  booktabs = TRUE,
  format = "latex"
) %>%
kableExtra::kable_styling(
  latex_options = c("striped", "hold_position"),
  full_width = FALSE
) %>%
kableExtra::pack_rows("Rejections", 2, 3) %>%
kableExtra::pack_rows("Model performance", 4, 6)

```

Table 1: QA/QC CH4 Flux Summary

Metric	Value
Total observations	273
Rejections	
Rejected — poor model fit (n)	0
Rejected — total (%)	0
Model performance	
Passed — Linear model (n)	273
Passed — HM81 model (n)	164
Passed — Polynomial model (n)	273

```

# R2 vs RMSE diagnostic plot
set.seed(42)
plot_data <- fluxes_flagged %>%
slice_sample(n = min(15000, nrow())) %>%
select(
  lin_r.squared, lin_RMSE, CH4_lin_pass,
  HM81_r.squared, HM81_RMSE, CH4_HM81_pass,
  poly_r.squared, poly_RMSE, CH4_poly_pass,
  CH4_neg_flux
) %>%
rename(
  lin_pass = CH4_lin_pass,
  HM81_pass = CH4_HM81_pass,
  poly_pass = CH4_poly_pass
)

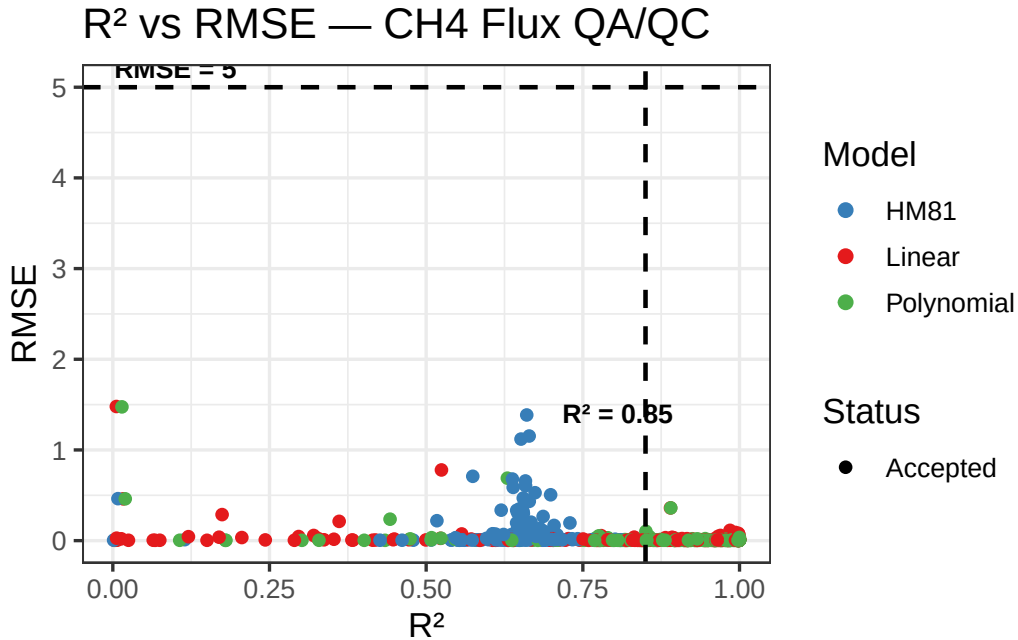
```

```

) %>%
pivot_longer(cols = -CH4_neg_flux, #cols = everything(),
names_to = c("model", ".value"),
names_pattern = "(lin|HM81|poly)_(.*)")
) %>%
rename(r2 = r.squared) %>%
mutate(
model = recode(model, lin = "Linear", HM81 = "HM81", poly = "Polynomial"),
status = case_when(
pass ~ "Accepted",
TRUE ~ "Rejected"
)
) %>%
filter(!is.na(r2), !is.na(RMSE))

QAQC_plot <- ggplot(plot_data, aes(x = r2, y = RMSE, color = model, shape = status)) +
geom_point(size = 2) +
geom_vline(xintercept = 0.85, linetype = "dashed", color = "black", linewidth = 0.8) +
geom_hline(yintercept = 5, linetype = "dashed", color = "black", linewidth = 0.8) +
annotate("text", x = 0.7, y = max(plot_data$RMSE, na.rm = TRUE) * 0.95,
label = "R2 = 0.85", hjust = -0.1, size = 3.5, fontface = "bold") +
annotate("text", x = 0.1, y = 5,
label = "RMSE = 5", vjust = -0.5, size = 3.5, fontface = "bold") +
scale_color_manual(values = c(
"Linear" = "#E41A1C",
"HM81" = "#377EB8",
"Polynomial" = "#4DAF4A"
)) +
scale_shape_manual(values = c(
"Accepted" = 16,
"Rejected" = 4
)) +
labs(
title = "R2 vs RMSE - CH4 Flux QA/QC",
x = "R2",
y = "RMSE",
color = "Model",
shape = "Status"
) +
theme_bw(base_size = 13) +
theme(legend.position = "right")
print(QAQC_plot)

```



##Adding manual flag to reject any fluxes that do not fit criteria

```
# adding manual flags to the ones that are crazy off based on visual checks
manual_reject <- c("2/24/2023_PTR_UP_6_101", #leak
                  "1/5/2023_OWC_UP_103_3", #Flat line with one low value in the middle
                  "1/5/2023_OWC_UP_107_123" #Flat line with one low value in the middle
)
fluxes_flagged <- fluxes_flagged %>%
mutate(
  CH4_rejected_manual = Plot %in% manual_reject,
  CH4_flux_rejected = CH4_rejected_model | CH4_rejected_manual)
```

##Do some visualization of QAQC plots to assess fluxes

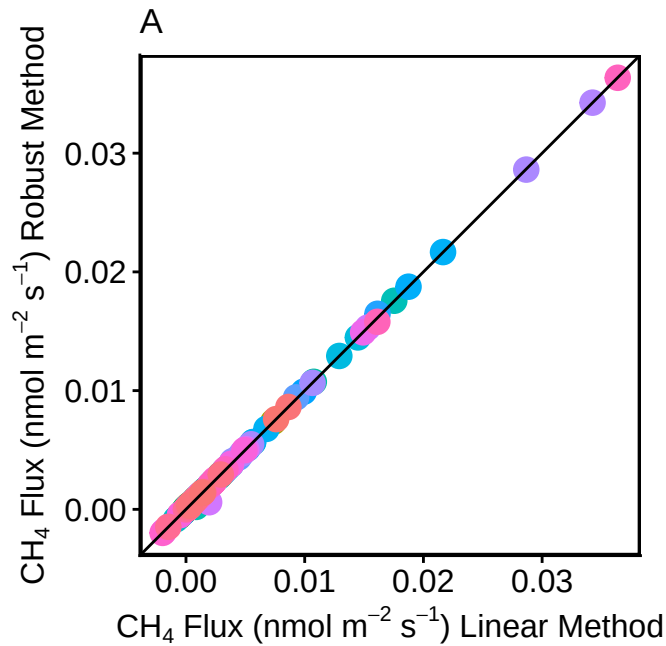
```
allflux_filtered <-fluxes_flagged%>%filter(!CH4_flux_rejected)

p1 <- ggplot(allflux_filtered, aes(lin_flux.estimate, rob_flux.estimate, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  labs(title = "A") +
  ylab(expression(paste( CH [4], " Flux (nmol m-2* " s-1*) Robust Method"))) +
  xlab(expression(paste( CH [4], " Flux (nmol m-2* " s-1*) Linear Method"))) +
```

```

guides(color = guide_legend(title = "Plot")) +
theme(legend.position="none") +
theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
      axis.title.y = element_text(size=12), legend.text=element_text(size=12),
      panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
      aspect.ratio = 1)
p1

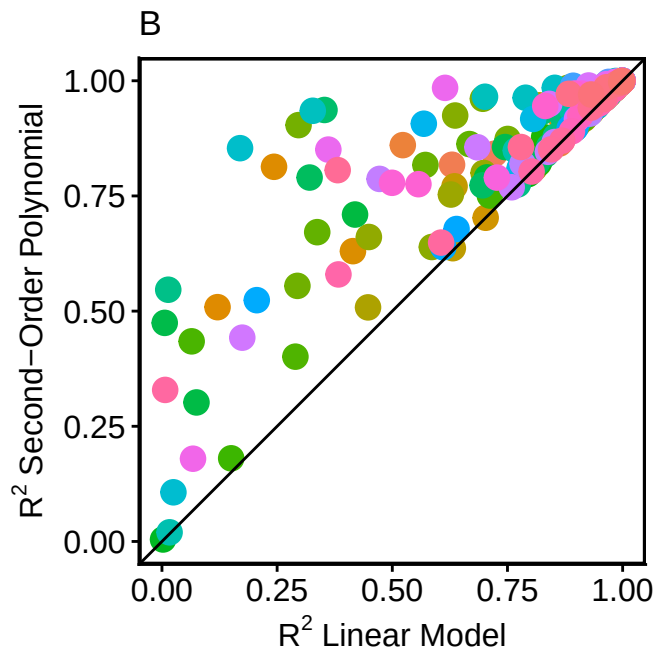
```



```

p2 <- ggplot(allflux_filtered, aes(lin_r.squared, poly_r.squared, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  xlim(0,1) + ylim(0,1) +
  labs(title = "B") +
  ylab(expression(paste("R"2* " Second-Order Polynomial")))) +
  xlab(expression(paste("R"2* " Linear Model")))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
p2

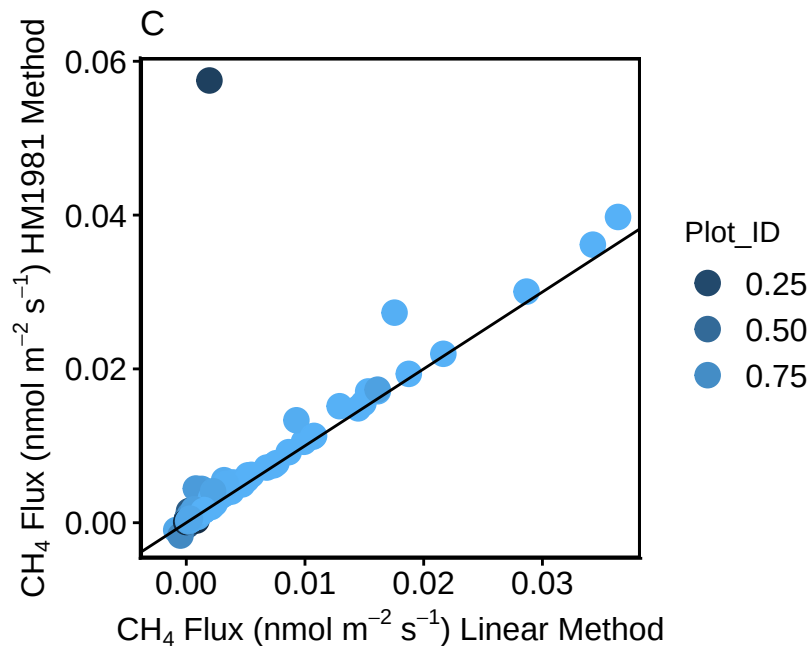
```



```
p3 <- ggplot(allflux_filtered, aes(lin_flux.estimate, HM81_flux.estimate, color = lin_r.squa
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  #xlim(0,50) + #ylim(0,50) +
  labs(title = "C") +
  ylab(expression(paste( CH [4], " Flux (nmol m-2* " s-1*) HM1981 Method"))) +
  xlab(expression(paste( CH [4], " Flux (nmol m-2* " s-1*) Linear Method"))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="right") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
```

p3

Warning: Removed 107 rows containing missing values or values outside the scale range (``geom_point()``).



```
#ggarrange(p1, p2, p3, ncol=3, nrow=1, common.legend = TRUE, legend="none")
```

18 Read out final dataframe and export for use elsewhere

```
allflux_filtered_renamed <- fluxes_flagged %>%
  mutate(
    Date = mdy(str_extract(Plot, "[0-9]+/[0-9]+/[0-9]{4}")),
    Month = month.abb[month(Date)],
    Site = str_extract(Plot, "(PTR|OWC|CRC)"),
    Zone = str_extract(Plot, "_TR_|_UP_") %>%
      str_remove_all("_"),
    Replicate = as.integer(
      str_extract(Plot, "(?<=_ (TR|UP)_)[0-9]+")
    ),
    Plot_ID = as.integer(
      str_extract(Plot, "[0-9]+$") ) %>%

# Optional: rebuild a clean Plot label
mutate(
  Plot_clean = paste(
```

```

        format(Date, "%Y-%m-%d"),
        Site,
        Zone,
        Replicate,
        Plot_ID,
        sep = "_"
    )
) %>%

select(
  Plot_clean,
  Date,
  Month,
  Site,
  Zone,
  Replicate,
  Plot_ID,
  everything(),
  -Plot
)

#export as a csv to the folder that you want
write.csv(allflux_filtered_renamed, file="Processed Data/COMPASS_Synoptic_LE_TreeStem_Fluxes.csv")
# #Change file name

```

19